When FLOPs Mislead: Benchmarking Neural Network Inference Energy on Apple Silicon

Aryan Shah
Texas Academy of Mathematics and Science
University of North Texas
Denton, TX, USA
aryanshah.2431@gmail.com

Romir Kadiam
Texas Academy of Mathematics and Science
University of North Texas
Denton, TX, USA
romirkadiam123@gmail.com

Abstract—Energy is a deployment constraint for edge-AI systems, but neural-network efficiency is still often discussed using architecture-level proxies such as FLOPs, parameter count, and model size. We benchmark five image-classification architectures on Apple Silicon under CPU-only PyTorch inference and compare measured per-inference energy, latency, FLOPs, and Energy-Delay Product (EDP). In the primary five-model mean table, latency almost completely explains between-model energy variation ($R^2 = 0.99999$), while FLOPs explains almost none ($R^2 = 0.00008$). EfficientNet-B0 consumes 2.65× the energy of ResNet-18 despite having 4.7× fewer FLOPs. A separate 50-window powermetrics audit on an Apple M4 Pro preserves the qualitative pattern: latency remains strongly predictive of measured energy ($R^2 = 0.980$), while FLOPs remains uninformative ($R^2 = 0.001$). We also summarize a residualized conditional WGAN-GP used only as supplemental support for sparse repeated-trial analysis. The results support a practical benchmarking lesson for embedded and edge inference: direct latency and power measurement are more informative than FLOP counts alone under a fixed runtime and input shape.

Index Terms—edge AI, benchmarking, neural network inference, energy measurement, Apple Silicon, energy-delay product

I. Introduction

High-performance embedded and edge systems increasingly run neural-network inference under strict energy and latency budgets. Model selection is often guided by FLOPs, parameter count, or model size because these quantities are easy to compute before deployment. They are useful descriptors, but they do not directly measure wall-clock work, memory traffic, kernel efficiency, or the behavior of a specific runtime on a specific processor [1]–[3].

This paper studies a narrow but practical question: for a fixed Apple Silicon CPU-only PyTorch runtime, which common architecture proxy best tracks measured inference energy? The answer in this benchmark is latency, not FLOPs. This is important for HPEC-style edge deployments because the relevant cost is the energy actually paid by the hardware under the deployment runtime, not the nominal arithmetic count of the neural network.

The contributions are:

- A five-architecture Apple-Silicon inference-energy benchmark comparing MobileNetV3-Small, MobileNetV2, ResNet-18, Tiny-ViT-5M, and EfficientNet-B0.
- A direct 50-window powermetrics audit with raw logs, per-window inference counts, confidence intervals, and environment metadata.
- A compact analysis showing that latency is a much stronger predictor of energy than FLOPs in both the original mean table and the later repeated audit.
- A scoped synthetic-trial modeling note: residualized conditional GANs can support exploratory repeated-trial analysis, but they should not replace independent power measurements.

The goal is not to crown one architecture as universally best. The benchmark is fixed-input and fixed-runtime. It asks what happens when common architecture families are deployed through the same CPU backend, and whether the usual complexity proxies preserve the measured energy ordering.

II. Related Work

Efficient neural-network design has produced architectures such as MobileNet, MobileNetV2, MobileNetV3, EfficientNet, SqueezeNet, and ShuffleNet, which reduce arithmetic through depthwise separable convolutions, inverted residuals, compound scaling, and channel grouping [4]–[9]. These designs often improve accuracy/FLOP tradeoffs, but hardware energy depends on more than FLOPs.

Prior systems work shows that data movement and memory hierarchy behavior can dominate arithmetic energy [1], [2]. Hardware-aware model design similarly emphasizes that latency and hardware utilization can diverge from FLOP counts [3], [10], [11]. Green AI and ML systems benchmarking further motivate reporting measured deployment costs rather than abstract complexity alone [12]–[15]. This paper adds a small, auditable Apple-Silicon case study centered on per-inference energy.

III. Methodology

A. Runtime and Hardware Scope

All primary results use CPU-only PyTorch inference with batch size 1 and input shape $1 \times 3 \times 224 \times 224$. CPU-

TABLE I
Direct repeated energy-audit environment.

Component	Setting
Device	MacBook Pro Mac16,8, Apple M4 Pro
CPU	12 cores: 8 performance, 4 efficiency
Memory	24 GB RAM
OS	macOS 26.2, build 25C56
Python / PyTorch	3.9.6 / 2.8.0 CPU float32
torchvision / timm	0.23.0 / 1.0.24
Threads	1 intra-op, 1 inter-op
Input	$1 \times 3 \times 224 \times 224$, batch 1
Power logging	powermetrics, 1 Hz
Window protocol	10 windows/model, 20 s/window
Order	randomized, seed 20260430

only execution was chosen to isolate a portable runtime path and avoid Neural Engine or Metal scheduling effects. It does not claim to be the most energy-efficient Apple Silicon backend.

The repository contains two energy sources. The first is the paper-aligned five-model mean table, which reports one mean latency and one mean energy per model from Apple-Silicon CPU inference measured with powermetrics at 1 Hz. The raw logs for that original mean table are unavailable, so we do not infer confidence intervals for it. The second is a later direct audit on a MacBook Pro Mac16,8 with an Apple M4 Pro, 24 GB RAM, macOS 26.2, Python 3.9.6, PyTorch 2.8.0, torchvision 0.23.0, and timm 1.0.24. The audit uses 10 20-second windows per model, 50 windows total, 1 Hz powermetrics sampling, 10 warmup inferences, and randomized model order.

The original mean table and the direct audit are intentionally kept separate. The first supports the main comparison used in the earlier manuscript; the second supplies raw logs and repeated-window uncertainty for a later local environment. Agreement between the two is therefore treated as robustness of the qualitative pattern rather than identity of absolute energy values.

B. Metrics

We report per-inference energy E in joules, latency L in milliseconds, and average power $P = 1000E/L$. Because power is sampled once per second, energy is interpreted as window-level energy divided by the number of inferences completed in the window. We also report Energy-Delay Product (EDP), a standard low-power metric:

$$\mathrm{EDP} = E \cdot L. \tag{1}$$

EDP is not accuracy-normalized; it measures energy-latency cost for fixed input-shape inference.

C. Benchmark Procedure

The measurement workflow is designed to separate model execution from power-log parsing. Each model is instantiated once, switched to evaluation mode, and run with gradients disabled. Warmup inferences are discarded so that module loading and one-time kernel setup do not dominate short windows. During each measured window, the script repeatedly executes forward passes until the window expires, records the number of completed inferences, and then divides the package-energy estimate for that same window by the inference count. The raw powermetrics output is retained, so the summary CSV is not the only source of evidence.

This windowed procedure is deliberately conservative for sub-100 ms inference. It does not try to attribute a single instantaneous power sample to one forward pass. Instead, it measures many repeated inferences under a controlled loop and reports the average energy paid by that loop. This matches the operating mode of many embedded inference services, where steady-state throughput and energy per request are more relevant than the energy of a single isolated call.

TABLE II
Benchmark validity checks used in the HPEC artifact.

Risk	Mitigation
Startup overhead	discard warmup inferences before each measured window
Order effects	randomize model order with a recorded seed
Power-log ambiguity	retain raw powermetrics logs and window counts
Proxy confusion	label latency-only sweep energy as proxy and exclude it from direct-energy claims
Synthetic overclaiming	evaluate WGAN-GP separately and treat it as supplemental only

Table II lists the main validity checks used when interpreting the benchmark. These checks are simple, but they make the artifact easier to audit: the paper distinguishes measured quantities from proxies, reports the exact hardware and software stack, and avoids using synthetic samples as physical measurements.

D. Supplemental Synthetic Modeling

The repository includes a conditional WGAN-GP pipeline used to explore synthetic repeated trials under sparse measurement budgets. An initial multi-device raw-scale model mode-collapsed. The revised model uses within-combination residuals: for each device/model combination c and feature x, the training value is

$$\tilde{x} = \mathrm{clip}\left((x - \mu_c)/\sigma_c, -3, 3\right). \tag{2}$$

Generated residuals are mapped back using the same μ_c and σ_c. This synthetic component is treated as supplemental. It can reproduce grounded means and rankings, but its spreads are not a substitute for held-out measured trial-level energy data.

The paper also includes a 17-architecture CPU-only latency sweep. That sweep has 30 measured latency trials per architecture and exact runtime metadata, but its energy and EDP columns are constant-power proxies

TABLE III
Paper-aligned five-model mean table. EDP is lower-is-better and is computed from measured energy and latency.

Model	FLOPs (G)	Lat. (ms)	Energy (J)	EDP (J·ms)
MobileNetV3-S	0.22	15.14	0.080	1.211
MobileNetV2	0.30	20.07	0.106	2.127
ResNet-18	1.82	20.68	0.110	2.275
Tiny-ViT-5M	1.30	41.33	0.219	9.051
EfficientNet-B0	0.39	55.08	0.292	16.083

TABLE IV
Univariate predictors of energy in the paper-aligned mean table ($n = 5$).

Predictor	R^2	Pearson r
Latency (ms)	0.99999	0.99999
FLOPs (G)	0.00008	-0.009
Parameters (M)	0.0547	0.234

rather than direct energy measurements. We use it only to contextualize latency behavior across additional small architectures; all energy claims in this paper come from the five-model measured energy tables.

IV. Results

Table III summarizes the five-model paper-aligned mean table. The key result is that energy follows latency rather than FLOPs. EfficientNet B0 has only 0.39 GFLOPs, but it has the largest latency and energy. ResNet-18 has 1.82 GFLOPs, yet consumes less than half the energy of EfficientNet-B0 in this setup.

The ordering is monotone in latency and not monotone in FLOPs. MobileNetV3 Small is the lowest-energy model and EfficientNet-B0 is the highest-energy model. ResNet-18 is the clearest counterexample to a FLOP-based story: it has the largest FLOP count in the set but remains close to MobileNetV2 in latency and energy. EfficientNet-B0 has substantially fewer FLOPs than ResNet-18, yet it takes longer and consumes more energy.

A. Predictor Analysis

We fit univariate linear models predicting energy from latency, FLOPs, and parameter count. Table IV shows that latency explains nearly all measured energy variation in the five-model mean table, while FLOPs and parameters do not. Leave-one-out checks preserve the same qualitative result: latency R^2 ranges from 0.99997 to 0.999999, while FLOPs ranges from 0.028 to 0.207.

B. Repeated Energy Audit

Table V reports the separate direct M4 Pro powermetrics audit. Absolute values differ from the original paper-aligned mean table because the audit was collected later on a specific local environment. It is therefore used as repeatability evidence, not as a retroactive uncertainty

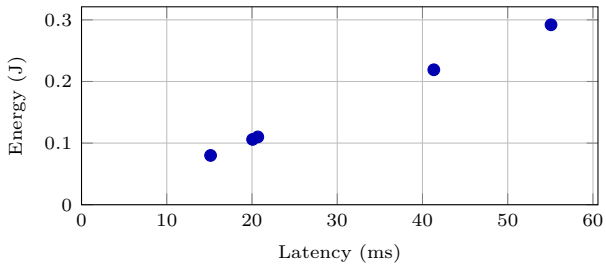

Fig. 1. Measured energy versus latency in the five-model mean table. Latency gives $R^2 = 0.99999$.

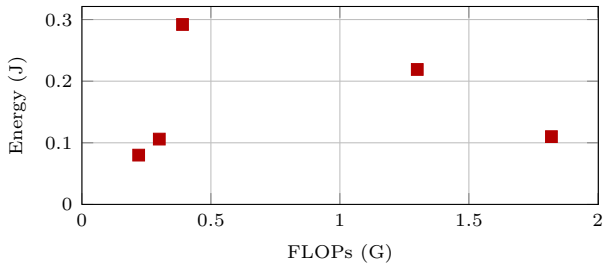

Fig. 2. Measured energy versus FLOPs in the same table. FLOPs gives $R^2 = 0.00008$ and ranks some models in the wrong direction.

estimate for the original table. Still, the same pattern holds: latency explains most between-model energy variation ($R^2 = 0.980$), and FLOPs remains weak ($R^2 = 0.001$).

The repeated audit exposes measurement variability that the original mean table cannot provide. The confidence intervals are widest for the models with longer inference windows and more variable power behavior. We keep all windows, including high-power windows, because removing them would hide deployment-relevant variability. The ranking remains qualitatively stable: MobileNetV3-Small, MobileNetV2, and ResNet-18 form a low-energy group, while Tiny-ViT-5M and EfficientNet-B0 are higher-energy under this CPU runtime.

C. Expanded Latency Sweep

The 17-architecture latency sweep broadens the performance context without making new direct energy claims. Table VI spans very small MLPs, patch mixers, plain CNNs, depthwise CNNs, grouped convolutions, residual CNNs, attention-gated CNNs, and ConvNeXt-like blocks. The sweep shows that latency can vary by orders of magnitude even when FLOPs are small, which is consistent with the direct-energy finding that runtime behavior matters.

This sweep is useful for systems interpretation. For example, the depthwise small model has fewer FLOPs than a tiny two-convolution CNN but higher latency, which is consistent with lower arithmetic intensity and less efficient CPU kernels. Patch-mixer models are also fast at low arithmetic counts, while the ConvNeXt-like micro model is slower than several architectures with larger FLOP counts. The result is not direct energy evidence,

TABLE V
Direct repeated M4 Pro energy audit, 10 windows per model.

Model	Energy (J)	95% CI (J)	Lat. (ms)	Power (W)
MobileNetV3-S	0.061	[0.043,0.080]	9.70	6.25
MobileNetV2	0.066	[0.053,0.079]	10.45	6.29
ResNet-18	0.074	[0.068,0.079]	10.34	7.10
Tiny-ViT-5M	0.176	[0.156,0.197]	25.00	7.04
EfficientNet-B0	0.206	[0.185,0.227]	33.76	6.09

but it helps explain why FLOPs alone should not be used to infer energy on this platform.

D. Synthetic Model Fidelity

The residualized WGAN-GP is evaluated only as a supplemental modeling tool. Table VII summarizes the paper-aligned synthetic fidelity against the grounded Apple-only distribution. Energy and latency pass Kolmogorov–Smirnov checks at $p > 0.05$, and the synthetic means preserve the measured energy ranking. The strongest claim is therefore limited: the synthetic model can reproduce the grounded distribution used to train it. It does not prove physical variance without independent repeated measurements.

V. Discussion

The main practical conclusion is not that FLOPs are useless. FLOPs still describe arithmetic scale, and they are often useful for first-pass model screening. The conclusion is that FLOPs are insufficient as an energy proxy for a specific deployment stack. On Apple Silicon CPU-only PyTorch inference, latency captures backend effects that FLOPs miss: memory traffic, kernel selection, vectorization, depthwise-convolution efficiency, attention implementation, and cache behavior.

The architecture comparisons illustrate this. MobileNet-style depthwise convolutions reduce arithmetic, but on a general-purpose CPU they can become memory-bound. EfficientNet-B0 has fewer FLOPs than ResNet-18, but its compound scaling and operator mix produce longer latency. Tiny-ViT-5M has fewer FLOPs than ResNet-18 but pays for attention and projection operations that are less friendly to this CPU runtime. These effects are familiar in hardware-aware model design, but this benchmark shows their impact on measured energy in a small Apple-Silicon edge-inference case study.

The synthetic GAN results should be interpreted cautiously. The residualized GAN reproduced measured means within a few percent in the repository workflow, but it was trained against grounded samples derived from measured means. This supports sensitivity analysis and artifact generation, not independent physical validation. HPEC-style benchmarking should prioritize direct power logs and only use synthetic trials as clearly labeled supplemental data.

The results suggest three practical rules for edge-inference benchmarking. First, report latency next to energy even when energy is the headline metric, because latency captures much of the backend behavior that FLOPs hide. Second, do not compare architectures using FLOPs alone when the operator mix changes: depthwise convolutions, attention blocks, and compound-scaled CNNs can have very different CPU efficiency at similar arithmetic counts. Third, preserve enough raw measurement state for others to audit the result. A single summary table is hard to diagnose; raw power logs, window durations, inference counts, thread settings, and hardware metadata make the benchmark substantially more useful.

For HPEC, the strongest aspect of the work is the benchmark artifact rather than the GAN. The raw audit logs, randomized model order, window counts, and environment JSON make the repeated energy audit inspectable. The GAN is still useful because it documents a failure mode: raw-scale multi-device training collapsed toward averages, while combo-normalized residual training preserved within-combination structure. That lesson is relevant for hardware-behavior modeling, where between-device variation can swamp the effect being studied. However, any deployment recommendation should be grounded in direct measurements.

VI. Deployment Implications

For an embedded developer choosing between candidate networks, the benchmark supports a measurement-first workflow. FLOPs can narrow the design space, but a small deployment harness should then measure latency and power under the exact runtime that will ship. This is especially important when candidates use different operator families. A dense residual CNN, a depthwise CNN, and a Transformer-like vision model can present similar arithmetic budgets to a spreadsheet while exercising the CPU backend very differently.

The result also affects how model-compression work should be reported. A compressed architecture with fewer FLOPs is not automatically a lower-energy architecture on a general-purpose CPU. If compression changes the operator mix toward kernels with poor vectorization or poor cache reuse, wall-clock time can increase enough to erase the arithmetic gain. Conversely, a model with more nominal arithmetic can be competitive if the runtime executes its kernels efficiently. This helps explain why ResNet-18 is not the highest-energy model in our five-model set despite having the most FLOPs.

For HPEC-style systems papers, this suggests that benchmark artifacts should include the runtime controls that determine whether results are comparable: thread count, batch size, input shape, warmup policy, sampling interval, hardware identifier, software versions, and whether accelerators were enabled. Without those controls, energy comparisons can become statements about

TABLE VI
Full 17-architecture direct latency sweep. Energy values in that sweep are excluded here because they are constant-power proxies, not direct powermetrics measurements.

Architecture	Family	Params (M)	FLOPs (G)	Lat. (ms)	P95 Lat. (ms)	CV
Global avg. MLP tiny	MLP	0.065	0.0001	0.026	0.026	0.013
Patch mixer small	Mixer	0.224	0.0419	0.465	0.528	0.071
Patch mixer medium	Mixer	0.435	0.1013	0.792	0.866	0.056
CNN tiny 2conv	Plain CNN	0.038	0.0398	0.793	0.836	0.029
CNN small 4conv	Plain CNN	0.112	0.1699	1.306	1.375	0.032
Depthwise small	Depthwise CNN	0.053	0.0233	1.446	1.508	0.027
Grouped conv CNN	Grouped CNN	0.154	0.1713	1.924	2.137	0.064
Depthwise medium	Depthwise CNN	0.112	0.0501	1.998	2.111	0.051
Attention gate CNN	Attention CNN	0.179	0.5260	2.373	2.428	0.014
CNN medium 6conv	Plain CNN	0.482	0.7463	2.416	2.497	0.020
Squeeze-expand CNN	Squeeze CNN	0.173	0.5967	3.114	3.219	0.016
Residual CNN small	Residual CNN	0.177	1.0623	3.331	3.484	0.038
Inverted residual small	Inverted residual	0.080	0.2792	3.366	3.463	0.019
Bottleneck residual	Bottleneck CNN	0.227	0.7244	3.480	3.639	0.026
Inverted residual wide	Inverted residual	0.167	0.8118	6.739	6.858	0.014
Residual CNN medium	Residual CNN	0.514	3.4142	6.848	8.531	0.173
ConvNeXt micro	ConvNeXt-like	0.240	0.4482	7.554	7.793	0.022

TABLE VII
Supplemental GAN fidelity against grounded Apple-only data.

Feature	Wasserstein	KS	p
Energy	0.0039	0.064	0.156
Latency	0.6934	0.067	0.120
Derived power	0.0457	0.047	0.498

TABLE VIII
External embedded reference points. Energy is included only when it can be inferred from a reported latency and an explicit published power figure.

Device	Model	Lat. (ms)	Energy (J)	Evidence
Coral TPU	MNetV2	2.6	0.0052	inferred
R-Pi 4	MNetV2	26.4	–	latency
R-Pi 4	RN-18	100.3	–	latency
STM32H747I	MNetV2	1118.27	–	latency

hidden runtime choices rather than about the architecture being evaluated.

A. External Embedded Reference Points

Table VIII gives non-Apple reference points from public embedded benchmarks: Coral Edge TPU [16], [17], PyTorch Raspberry Pi [18], and STM32 AI Model Zoo [19]. These rows are not used in the Apple-Silicon regression and should not be read as direct measurements from our artifact.

The contrast is useful but limited. Coral's inferred energy uses a published 2 W Edge TPU figure and benchmark latency, while the Raspberry Pi and STM32 rows are latency-only. They therefore strengthen the motivation for hardware-specific measurement but do not replace our direct Apple-Silicon power logs.

VII. Limitations

The study is intentionally narrow. It covers five architectures, one input shape, batch size 1, and CPU-only PyTorch inference. It does not evaluate Apple Neural Engine, Metal Performance Shaders, GPUs, quantization, or accuracy-normalized energy. The original five-model mean table lacks raw power logs; the repository adds a later direct audit with raw logs, but that audit is not the original source run. Finally, 1 Hz power sampling is coarse relative to per-inference latency, so energy is attributed over windows rather than instantaneous inference events.

There are also statistical limits. The main predictor analysis has only five architectures, so exact R^2 values should be read as descriptive statistics for this benchmark rather than universal constants. We include leave-one-out checks, rank checks, and a separate audit, but a stronger HPEC version would add more direct-energy architectures, hardware backends, and batch sizes. The current result is best viewed as a reproducible case study showing why FLOP counts can mislead under one realistic edge-inference stack.

VIII. Artifact Availability

The repository includes source code, tables, scripts, raw audit logs, and metadata at https://github.com/jubs-2431/which-neural-networks-waste-the-most-energy. Key artifacts are the Apple benchmark CSV, the M4 Pro energy-summary CSV, raw powermetrics logs, the measurement script, and the measurement protocol.

The intended reproduction path is lightweight. The existing CSV artifacts can be inspected without rerunning privileged power measurement. Repeating the direct energy audit on macOS requires powermetrics, which asks for administrator privileges, and writes a new set of window-level logs. The expanded latency sweep can be rerun without privileged access and provides a sanity check that

the CPU-only runtime and thread settings are configured as expected. The paper source and compiled HPEC draft are included so that the tables in this submission can be traced directly back to repository artifacts.

IX. Conclusion

In this Apple-Silicon CPU-only inference benchmark, measured energy tracks latency far more closely than FLOPs. The result is visible in both the paper-aligned five-model mean table and a later 50-window direct energy audit. For edge-AI and embedded deployments, the engineering lesson is direct: benchmark the deployed runtime with auditable power traces, and treat FLOPs as an incomplete proxy rather than an energy measurement.

References

[1] M. Horowitz, "Computing's energy problem (and what we can do about it)," IEEE Micro, vol. 34, no. 5, pp. 10–14, 2014.

[2] Y.-H. Chen et al., "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," IEEE J. Solid-State Circuits, 2017.

[3] Y. He et al., "Understanding FLOPs is not enough: A study of hardware-aware neural networks," arXiv preprint arXiv:1912.01090, 2019.

[4] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint arXiv:1704.04861, 2017.

[5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2018.

[6] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE Int. Conf. Computer Vision, 2019.

[7] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. Int. Conf. Machine Learning, 2019.

[8] F. N. Iandola et al., "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size," arXiv:1602.07360, 2016.

[9] X. Zhang et al., "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in Proc. IEEE CVPR, 2018.

[10] M. Tan et al., "MnasNet: Platform-aware neural architecture search for mobile," in Proc. IEEE CVPR, 2019.

[11] B. Wu et al., "FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search," in Proc. IEEE CVPR, 2019.

[12] R. Schwartz et al., "Green AI," Communications of the ACM, vol. 63, no. 12, pp. 54–63, 2020.

[13] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in Proc. ACL, 2019.

[14] D. Patterson et al., "Carbon emissions and large neural network training," arXiv:2104.10350, 2021.

[15] V. J. Reddi et al., "MLPerf inference benchmark," in Proc. ACM/IEEE ISCA, 2020.

[16] Google Coral, "Edge TPU performance benchmarks," accessed May 2026.

[17] Google Coral, "Frequently asked questions," accessed May 2026.

[18] PyTorch, "Real time inference on Raspberry Pi," accessed May 2026.

[19] STMicroelectronics, "STM32 AI model zoo: MobileNetV2," accessed May 2026.

[20] I. Gulrajani et al., "Improved training of Wasserstein GANs," in Advances in Neural Information Processing Systems, 2017.